EXP:10 MINI PROJECT

CODE:

```
import streamlit as st
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    ConfusionMatrixDisplay
)

st.set_page_config(
    page_title="Iris AI Analytics",
    page_icon="🌸",
    layout="wide"
)

st.markdown("""
```

```
<style>
.stApp {
  background: linear-gradient(135deg, #101323, #171c30);
  color: #f4f6ff;
}
h1, h2, h3 {
  color: #b9c7ff;
}
[data-testid="stMetric"] {
  background: #222941;
  padding: 18px;
  border-radius: 14px;
  border: 1px solid #394363;
}
div.stButton > button {
  background: linear-gradient(90deg, #7667ff, #43c6ac);
  color: white;
  border: none;
  border-radius: 10px;
  font-weight: bold;
  padding: 10px 22px;
}
</style>
""", unsafe_allow_html=True)

@st.cache_resource
def train_models():
  iris = load_iris()
  X, y = iris.data, iris.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
  X, y, test_size=0.2, random_state=42, stratify=y
)

models = {
  "SVM": make_pipeline(
    StandardScaler(), SVC(probability=True, random_state=42)
  ),
  "Decision Tree": DecisionTreeClassifier(
    max_depth=4, random_state=42
  ),
  "Random Forest": RandomForestClassifier(
    n_estimators=100, random_state=42
  )
}

metrics = {}
predictions = {}

for name, model in models.items():
  model.fit(X_train, y_train)
  pred = model.predict(X_test)
  predictions[name] = pred

  metrics[name] = {
    "Accuracy": accuracy_score(y_test, pred) * 100,
    "Precision": precision_score(
      y_test, pred, average="weighted", zero_division=0
    ) * 100,
    "Recall": recall_score(
```

```
            y_test, pred, average="weighted", zero_division=0
        ) * 100,
        "F1-score": f1_score(
            y_test, pred, average="weighted", zero_division=0
        ) * 100
    }

  return iris, models, metrics, X_test, y_test, predictions

iris, models, metrics, X_test, y_test, predictions = train_models()

df = pd.DataFrame(metrics).T
best_model = df["Accuracy"].idxmax()

st.title("🌸 IRIS AI ANALYTICS")
st.markdown(
  "### Machine Learning Performance & Prediction Dashboard"
)
st.caption(
  "CLASSIFICATION • MODEL EVALUATION • PREDICTIVE ANALYTICS"
)

st.divider()

c1, c2, c3, c4 = st.columns(4)

c1.metric("🧠 Models Trained", len(models))
c2.metric("📊 Test Samples", len(y_test))
c3.metric("🏆 Best Model", best_model)
```

```
c4.metric("🎯 Best Accuracy", f"{df['Accuracy'].max():.2f}%")

st.divider()

st.subheader("📋 Model Performance Summary")

display_df = df.round(2)
display_df.index.name = "Algorithm"

st.dataframe(
  display_df.style.highlight_max(
    axis=0, color="#285b50"
  ).format("{:.2f}%"),
  use_container_width=True
)

st.success(
  f"🏆 Current leader: {best_model} "
  f"with {df.loc[best_model, 'Accuracy']:.2f}% test accuracy."
)

st.divider()
st.subheader("📈 Visualization 1 — Accuracy Leaderboard")
st.write("Compare the test accuracy achieved by each trained algorithm.")

fig1, ax1 = plt.subplots(figsize=(9, 4))

names = df.index.tolist()
values = df["Accuracy"].values
```

```
colors = ["#42c6ac" if n == best_model else "#7777ff" for n in names]

bars = ax1.barh(names, values, color=colors, height=0.55)

ax1.set_xlim(0, 105)
ax1.set_xlabel("Accuracy (%)")
ax1.set_title("Model Accuracy Comparison")
ax1.grid(axis="x", linestyle="--", alpha=0.25)
ax1.set_axisbelow(True)

for bar, value in zip(bars, values):
    ax1.text(
        value + 1,
        bar.get_y() + bar.get_height() / 2,
        f"{value:.2f}%",
        va="center",
        fontweight="bold"
    )

fig1.tight_layout()
st.pyplot(fig1)
plt.close(fig1)

st.divider()
st.subheader(" 📊 Visualization 2 — Metrics Comparison")
st.write(
    "Compare Precision, Recall and F1-score alongside accuracy. "
    "These metrics help evaluate classification performance."
)
```

```
metric_names = ["Accuracy", "Precision", "Recall", "F1-score"]
x = np.arange(len(metric_names))
width = 0.24

fig2, ax2 = plt.subplots(figsize=(10, 5))

colors2 = ["#7777ff", "#42c6ac", "#ffb454"]

for i, name in enumerate(names):
  vals = [metrics[name][m] for m in metric_names]
  ax2.bar(
    x + (i - 1) * width,
    vals,
    width,
    label=name,
    color=colors2[i]
  )

ax2.set_ylabel("Score (%)")
ax2.set_title("Classification Metrics by Model")
ax2.set_xticks(x)
ax2.set_xticklabels(metric_names)
ax2.set_ylim(0, 110)
ax2.legend()
ax2.grid(axis="y", linestyle="--", alpha=0.25)
ax2.set_axisbelow(True)

fig2.tight_layout()
st.pyplot(fig2)
```

```
plt.close(fig2)


st.divider()
st.subheader("🔮 Try Your Own Prediction")


selected = st.selectbox(
    "Choose a trained model",
    list(models.keys()),
    index=list(models.keys()).index(best_model)
)


col1, col2 = st.columns(2)


with col1:
    sl = st.slider("Sepal length (cm)", 4.0, 8.0, 5.1)
    sw = st.slider("Sepal width (cm)", 2.0, 4.5, 3.5)


with col2:
    pl = st.slider("Petal length (cm)", 1.0, 7.0, 1.4)
    pw = st.slider("Petal width (cm)", 0.1, 2.5, 0.2)


if st.button("🚀 Predict Species", type="primary"):
    sample = [[sl, sw, pl, pw]]
    model = models[selected]
    pred = int(model.predict(sample)[0])


    st.success(
        f"🌸 Predicted Species: {iris.target_names[pred].title()}"
    )
```

```
    if hasattr(model, "predict_proba"):
        probabilities = model.predict_proba(sample)[0]
        st.metric(
            "Prediction confidence",
            f"{probabilities[pred] * 100:.2f}%"
        )

        prob_df = pd.DataFrame({
            "Species": [
                name.title() for name in iris.target_names
            ],
            "Probability (%)": probabilities * 100
        }).set_index("Species")

        st.bar_chart(prob_df)

st.divider()
st.subheader("🎯 Confusion Matrix")

matrix_model = st.selectbox(
    "Select model for confusion matrix",
    list(models.keys()),
    key="matrix_model"
)

fig3, ax3 = plt.subplots(figsize=(7, 4))

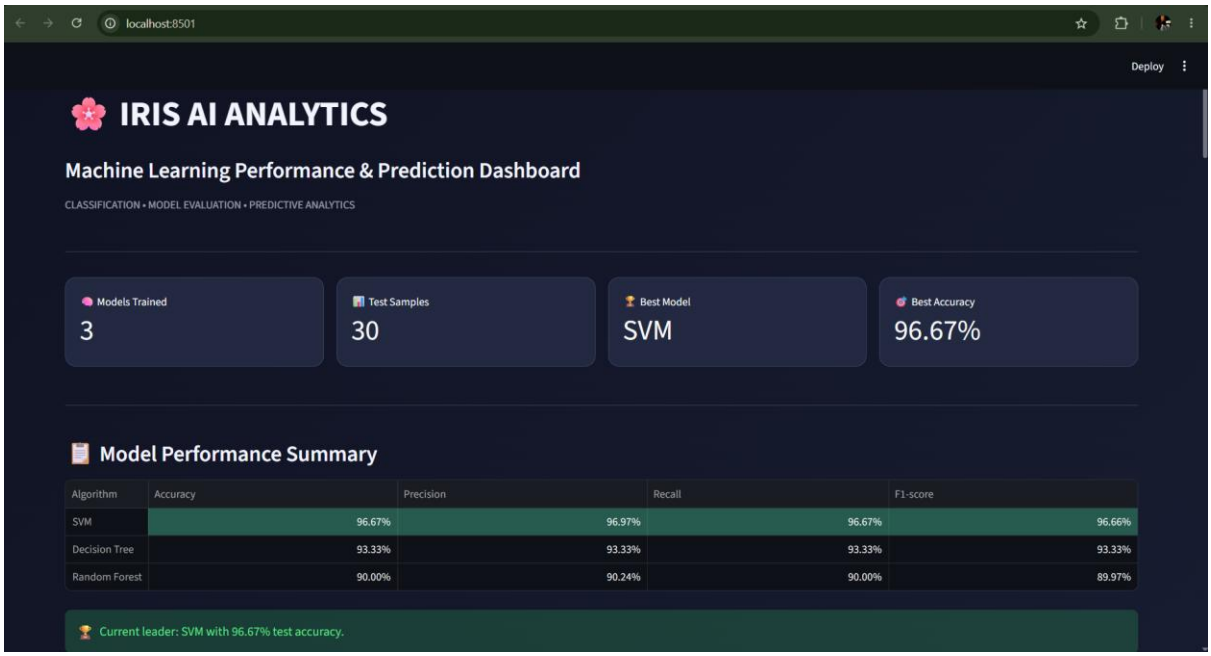
ConfusionMatrixDisplay.from_predictions(
```

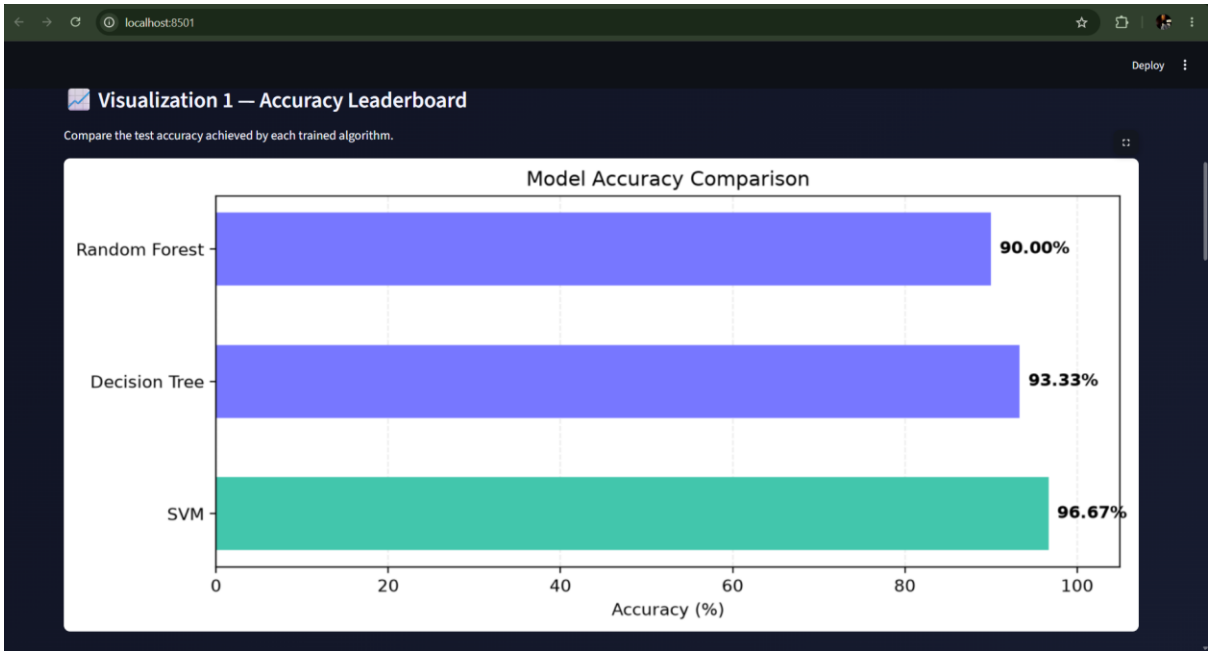
```
    y_test,
    predictions[matrix_model],
    display_labels=iris.target_names,
    cmap="Blues",
    ax=ax3,
    colorbar=False
)

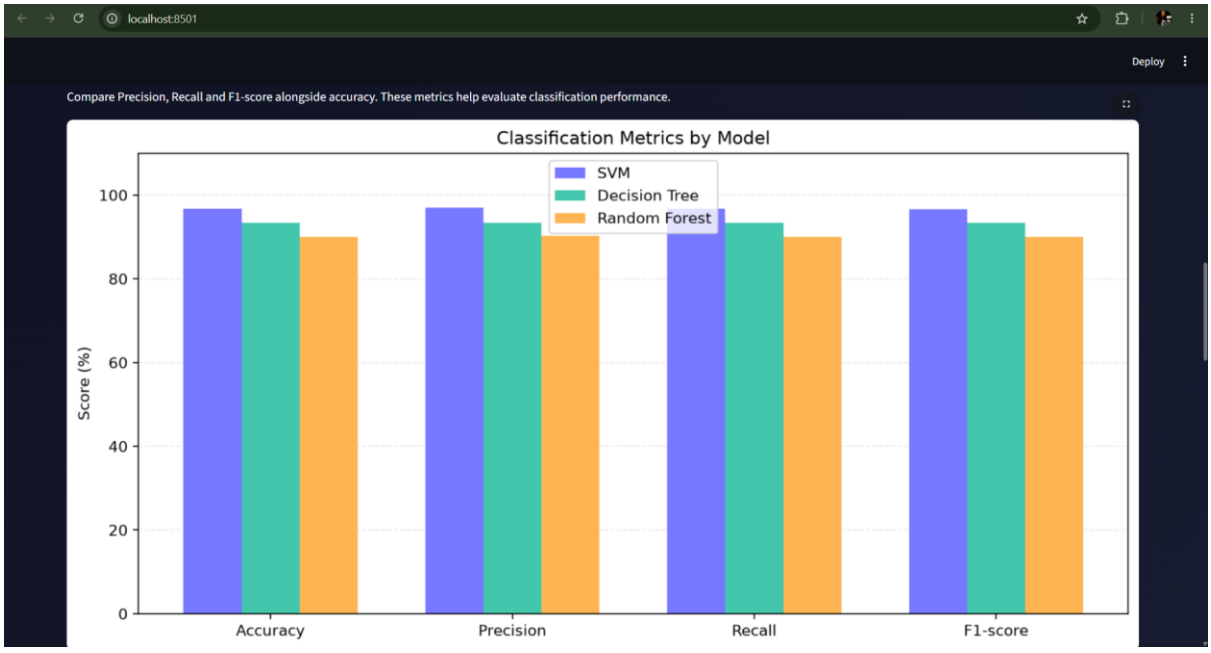
ax3.set_title(f"{matrix_model} — Test Results")
fig3.tight_layout()
st.pyplot(fig3)
plt.close(fig3)

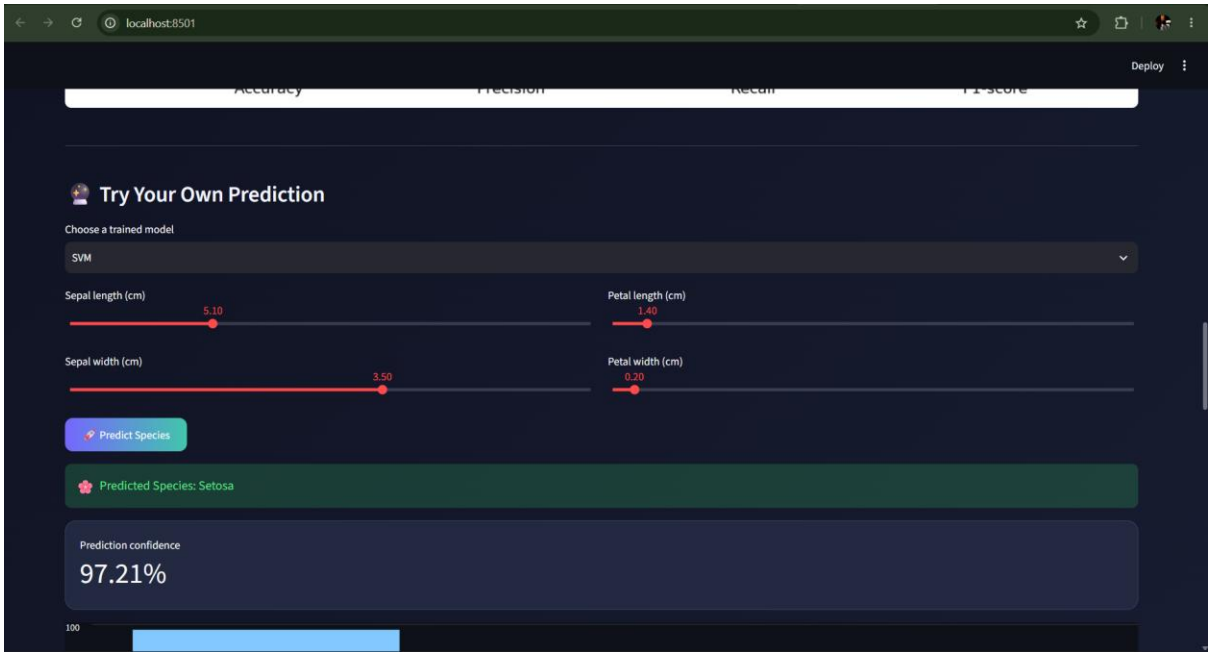
st.divider()
st.caption(
    "Built with Python • Scikit-learn • Pandas • Matplotlib • Streamlit"
)
```

OUTPUT:




localhost:8501
Deploy
Visualization 1 — Accuracy Leaderboard
Compare the test accuracy achieved by each trained algorithm.
Model Accuracy Comparison
Random Forest
Decision Tree
SVM
90.00%
93.33%
96.67%
0
20
40
60
80
100
Accuracy (%)


localhost:8501
Deploy
Compare Precision, Recall and F1-score alongside accuracy. These metrics help evaluate classification performance.
Classification Metrics by Model
SVM
Decision Tree
Random Forest
Score (%)
0
20
40
60
80
100
Accuracy
Precision
Recall
F1-score


localhost:8501
Deploy
Try Your Own Prediction
Choose a trained model
SVM
Sepal length (cm)
5.10
Petal length (cm)
1.40
Sepal width (cm)
3.50
Petal width (cm)
0.20
Predict Species
Predicted Species: Setosa
Prediction confidence
97.21%
100